BEM Solver для электромагнитного рассеяния

Полное руководство: теория, реализация, ускорение, валидация

Содержание

1 Введение **4**
- 1.1 Основные возможности . . . 4
- 1.2 Состав проекта . . . 4
- 1.3 Зависимости . . . 4

2 Физика задачи **5**
- 2.1 Постановка: рассеяние плоской волны на частице . . . 5
- 2.2 Параметры среды . . . 5
- 2.3 Интегральные операторы . . . 5
- 2.4 Почему скалярное ядро, а не тензорное? . . . 6
- 2.5 PMCHWT-формулировка . . . 7
- 2.6 Случай PEC (EFIE) . . . 7
- 2.7 Случай поглощающих частиц . . . 7

3 Дискретизация: RWG базисные функции **7**
- 3.1 Определение . . . 7
- 3.2 Поверхностная дивергенция . . . 8
- 3.3 Масштаб задачи: icosphere . . . 8
- 3.4 Выбор разрешения сетки . . . 8

4 Загрузка произвольных сеток **9**

5 Адаптивное уточнение сетки **9**
- 5.1 Red-refinement с green closure . . . 9
- 5.2 Автоматическое уточнение по длине волны . . . 9

6 Сборка матрицы: операторы L и K **10**
- 6.1 Дискретный оператор L . . . 10
- 6.2 Дискретный оператор K . . . 10
- 6.3 Эксплуатация симметрии (3 прохода вместо 4) . . . 10
- 6.4 Батчевая сборка . . . 11

7 Обработка сингулярностей **11**
7.1 Извлечение сингулярности (Graglia 1993) [2,14] . . . 11
7.2 Аналитический интеграл потенциала . . . 11
7.3 Оператор K: PV-интеграл . . . 11
7.4 Квадратуры Dunavant . . . 11

8 Сборка PMCHWT системы **12**
8.1 Параллельная сборка . . . 12
8.2 Правая часть . . . 12
8.3 Пример: полный цикл решения . . . 12

9 Multi-body BEM (кластеры частиц) **13**
9.1 Принцип . . . 13
9.2 API . . . 13

10 Решение системы **14**
10.1 Прямой метод: LU-разложение . . . 14
10.2 GMRES (итерационный) . . . 14
10.3 H-матричный GMRES . . . 14
10.4 FMM-ускоренный GMRES . . . 15

11 Дальнее поле и сечения **15**
11.1 Амплитуда рассеяния . . . 15
11.2 Сечения рассеяния и экстинкции . . . 15
11.3 Поглощение . . . 16

12 Матрица амплитудного рассеяния и матрица Мюллера **16**
12.1 Амплитудная матрица . . . 16
12.2 Матрица Мюллера . . . 16
12.3 Пример: вычисление матрицы Мюллера . . . 16

13 Усреднение по ориентациям **17**
13.1 Теория . . . 17
13.2 Почему матрицу Z не нужно пересобирать . . . 17
13.3 Наивная vs батчевая реализация . . . 17

14 OpenCL-ускорение **18**
14.1 Зачем float32? . . . 18
14.2 Ядро сборки `assemble_LK_block` . . . 18
14.3 Ядро matvec `complex_matvec` . . . 19
14.4 Сингулярные коррекции на CPU . . . 19
14.5 Производительность (ref=3, 1920 RWG) . . . 19
14.6 Установка OpenCL . . . 19
14.7 Использование . . . 19

15 Валидация **20**
15.1 Диэлектрическая сфера vs Mie ($m = 1.5$, реальный) . . . 20
15.2 PEC сфера vs Mie . . . 20
15.3 Поглощающие частицы ($m \in \mathbb{C}$) . . . 20
15.4 Оптическая теорема . . . 20

15.5 Матрица Мюллера 20
15.6 Несферические частицы 21

16 Масштабирование BEM vs DDA **21**

17 Исправленные ошибки **21**

18 Ограничения и перспективы **22**
18.1 Текущие ограничения 22
18.2 Что реализовано (бывшие ограничения) 22

19 Литература **22**
19.1 Основные источники 22
19.2 PMCHWT и интегральные уравнения 23
19.3 Сингулярные интегралы и квадратуры 24
19.4 Быстрые алгоритмы (FMM, H-матрицы) 24
19.5 Рассеяние света, теория Ми, матрица Мюллера 25
19.6 Вычислительные аспекты и GPU 26

1 Введение

Проект реализует **метод граничных элементов** (Boundary Element Method, BEM) для решения задачи рассеяния электромагнитных волн на диэлектрических и идеально проводящих (PEC) телах произвольной формы. Реализация — чистый Python (NumPy/SciPy) с опциональными ускорениями через Numba (JIT) и OpenCL (GPU/CPU).

Используется физическая конвенция $e^{+i\omega t}$ и функция Грина $G = e^{ikR}/(4\pi R)$ [4,13].

1.1 Основные возможности

- PMCHWT-формулировка для диэлектрических тел + EFIE для PEC
- RWG базисные функции на треугольной поверхностной сетке
- Аналитическая обработка сингулярностей (Graglia 1993)
- Матрица Мюллера и усреднение по ориентациям (углы Эйлера)
- Мульти-тело BEM для кластеров частиц
- Адаптивное уточнение сетки (red/green closure)
- Квадратуры Dunavant 4, 7, 13, 25 точек
- FMM и H-матричное сжатие для $O(N \log N)$ matvec
- OpenCL float32 ядра для GPU-ускорения
- Numba JIT для сингулярных коррекций
- Валидация: PEC, диэлектрик, поглощающие ($m \in \mathbb{C}$), несферические

1.2 Состав проекта

Файл	Описание
`bem_core.py`	Ядро: меш, RWG, L/K, PMCHWT, дальнее поле, Мюллер, FMM, H-матрицы
`bem_opencl.py`	OpenCL float32 ядра сборки и GMRES matvec
`tests/`	39 тестовых скриптов
`scripts/`	Графики сравнения BEM/ADDA/Mie
`pdf/`	Данный мануал и графики результатов

1.3 Зависимости

Пакет	Статус	Назначение
`numpy`, `scipy`	обязательные	линейная алгебра, квадратуры, GMRES
`numba`	рекомендуется	JIT сингулярных коррекций (∼100× ускорение)
`pyopencl`	опционально	GPU/CPU OpenCL сборка и matvec
`matplotlib`	опционально	графики

Установка:

```
pip install numpy scipy numba
pip install pyopencl   #        OpenCL
```

2 Физика задачи

2.1 Постановка: рассеяние плоской волны на частице

Однородная диэлектрическая частица с относительным показателем преломления $m = n_1/n_0$ (может быть комплексным: $m = n + i\kappa$, где $\kappa > 0$ — поглощение) в однородной среде.

Падающая плоская волна:

$$\mathbf{E}^{\text{inc}}(\mathbf{r}) = \mathbf{E}_0\, e^{ik_{\text{ext}}\hat{\mathbf{k}}\cdot\mathbf{r}}, \qquad \mathbf{H}^{\text{inc}}(\mathbf{r}) = \frac{\hat{\mathbf{k}} \times \mathbf{E}_0}{\eta_{\text{ext}}}\, e^{ik_{\text{ext}}\hat{\mathbf{k}}\cdot\mathbf{r}} \tag{1}$$

где $k_{\text{ext}} = \omega\sqrt{\mu_0\varepsilon_{\text{ext}}}$, $\eta_{\text{ext}} = \sqrt{\mu_0/\varepsilon_{\text{ext}}}$.

2.2 Параметры среды

Параметр	Формула	Описание
k_{ext}	$2\pi/\lambda$	Волновое число среды
k_{int}	$m \cdot k_{\text{ext}}$	Волновое число внутри частицы
η_{ext}	1.0 (нормировка) или 377 Ом	Импеданс среды
η_{int}	η_{ext}/m	Импеданс внутри частицы (при $\mu_r = 1$)
ka	$k_{\text{ext}} \cdot a$	Размерный параметр
m	$n + i\kappa$	Комплексный показатель преломления

2.3 Интегральные операторы

Из уравнений Максвелла и теоремы эквивалентности следуют операторы электрического ($\mathcal{L}$) и магнитного ($\mathcal{K}$) типов:

$$\mathcal{L}_k[\mathbf{X}](\mathbf{r}) = ik \int_S G_k\, \mathbf{X}\, dS' + \frac{i}{k}\nabla \int_S G_k\, \nabla' \cdot \mathbf{X}\, dS' \tag{2}$$

$$\mathcal{K}_k[\mathbf{X}](\mathbf{r}) = \int_S \nabla G_k \times \mathbf{X}\, dS' \tag{3}$$

где $G_k(\mathbf{r}, \mathbf{r}') = e^{ik|\mathbf{r}-\mathbf{r}'|}/(4\pi|\mathbf{r}-\mathbf{r}'|)$ — скалярная функция Грина свободного пространства.

Физический смысл:

- $\mathcal{L}$ — связан с электрическим полем, создаваемым токами на поверхности (EFIE)
- $\mathcal{K}$ — связан с магнитным полем (MFIE)

2.4 Почему скалярное ядро, а не тензорное?

На первый взгляд кажется, что скалярная функция Грина $G = e^{ikR}/(4\pi R)$ не может менять поляризацию рассеянного поля: она умножает вектор тока на скаляр, сохраняя его направление. Однако операторы $\mathcal{L}$ и $\mathcal{K}$ на самом деле **тензорные**, несмотря на скалярную G.

Диадная функция Грина. Уравнения Максвелла для рассеянного поля имеют решение через **диадную** (тензорную) функцию Грина [13, глава 8]:

$$\bar{\bar{G}}(\mathbf{r}, \mathbf{r}') = \left(\bar{\bar{I}} + \frac{\nabla\nabla}{k^2} \right) G(\mathbf{r}, \mathbf{r}') \tag{4}$$

где $\bar{\bar{I}}$ — единичный тензор (3×3 единичная матрица), а $\nabla\nabla G$ — тензор второго ранга с компонентами $\partial^2 G/\partial x_i \partial x_j$.

Именно член $\nabla\nabla G/k^2$ перемешивает компоненты поляризации: даже если ток $\mathbf{J}$ направлен по x, результат $\bar{\bar{G}} \cdot \mathbf{J}$ имеет компоненты по x, y и z.

Связь с оператором $\mathcal{L}$. Действие диадной функции Грина на ток:

$$\bar{\bar{G}} \cdot \mathbf{J} = G \cdot \mathbf{J} + \frac{1}{k^2} \nabla(\nabla \cdot [G\mathbf{J}]) \tag{5}$$

После интегрирования по поверхности и интегрирования по частям (перебрасывания ∇' на $\mathbf{J}$) получаем в точности оператор $\mathcal{L}$:

$$\mathcal{L}[\mathbf{J}] = ik \underbrace{\int_S G\, \mathbf{J}\, dS'}_{\text{векторный потенциал } \mathbf{A}} + \frac{i}{k} \underbrace{\nabla \int_S G\, \nabla' \cdot \mathbf{J}\, dS'}_{\text{градиент скалярного потенциала } -\nabla\Phi} \tag{6}$$

- **Первый член** ($\sim \mathbf{A}$): скалярная G умножает вектор $\mathbf{J}$; поляризация сохраняется.
- **Второй член** ($\sim -\nabla\Phi$): интеграл $\int G\, \nabla' \cdot \mathbf{J}\, dS'$ — скаляр (потенциал Φ), но его **градиент** направлен произвольно — не вдоль $\mathbf{J}$, а вдоль $\nabla_{\mathbf{r}}$ в точке наблюдения. Именно этот член вносит тензорную структуру и перемешивает компоненты поля.

Таким образом, записывая ядро через скалярную G вместо тензорной $\bar{\bar{G}}$, мы не теряем физику — тензорная структура перенесена на дифференциальные операторы (∇, $\nabla\cdot$), а после дискретизации методом Галёркина — на базисные функции и их дивергенции.

Оператор $\mathcal{K}$ — явное вращение поляризации. Оператор

$$\mathcal{K}[\mathbf{J}] = \int_S \nabla G \times \mathbf{J}\, dS' \tag{7}$$

содержит **векторное произведение** $\nabla G \times \mathbf{J}$, которое по определению поворачивает вектор: результат перпендикулярен и ∇G, и $\mathbf{J}$. Если $\mathbf{J} \| \hat{\mathbf{x}}$, а $\nabla G \| \hat{\mathbf{z}}$, то $\nabla G \times \mathbf{J} \| \hat{\mathbf{y}}$ — поляризация повернулась на 90°. Кроме того, $\mathcal{K}$ связывает электрические токи с магнитным полем и наоборот (через PMCHWT), обеспечивая полный механизм изменения поляризации при рассеянии.

Переход к дискретной форме. После проекции на RWG-базис (метод Галёркина [1,13]) тензорный член $\nabla\nabla G$ превращается в скалярный div-div:

$$\int_S\int_S \mathbf{f}_m \cdot \left(\frac{\nabla\nabla G}{k^2}\right) \cdot \mathbf{f}_n \, dS \, dS' \;=\; \frac{1}{k^2}\int_S\int_S (\nabla_s \cdot \mathbf{f}_m)(\nabla'_s \cdot \mathbf{f}_n)\, G \, dS \, dS' \tag{8}$$

Тензорная структура «спрятана» в дивергенциях базисных функций $\nabla_s \cdot \mathbf{f}_n = \pm l_n / A_n^{\pm}$, которые являются скалярами. Благодаря этому все интегралы в коде сводятся к скалярной G — вычислительно эффективно и без потери физики.

2.5 PMCHWT-формулировка

Формулировка Poggio–Miller–Chang–Harrington–Wu–Tsai [3,9,10] из условия непрерывности тангенциальных полей на границе частицы:

$$\boxed{\begin{pmatrix} \eta_1\mathcal{L}_1 + \eta_2\mathcal{L}_2 & -(\mathcal{K}_1 + \mathcal{K}_2) \\ \mathcal{K}_1 + \mathcal{K}_2 & \dfrac{\mathcal{L}_1}{\eta_1} + \dfrac{\mathcal{L}_2}{\eta_2} \end{pmatrix} \begin{pmatrix} \mathbf{J} \\ \mathbf{M} \end{pmatrix} = + \begin{pmatrix} \mathbf{b}_E \\ \mathbf{b}_H \end{pmatrix}} \tag{9}$$

Индекс 1 = ext (среда), 2 = int (частица).

Критически важные знаки K-блоков: верхний правый $-K$, нижний левый $+K$, правая часть $+\mathbf{b}$. Эта асимметричная конвенция согласована с дальним полем через $s_M = -1$. Ошибка в знаках K-блоков — одна из самых распространённых ошибок в реализациях PMCHWT, приводящая к 10–50% ошибкам.

2.6 Случай PEC (EFIE)

Для идеально проводящего тела: $\mathbf{M} = 0$ (нет магнитного тока), система упрощается до EFIE:

$$\eta\, \mathcal{L}_{\text{ext}}[\mathbf{J}] = \mathbf{b}_E \tag{10}$$

2.7 Случай поглощающих частиц

При комплексном $m = n + i\kappa$ ($\kappa > 0$) волновое число внутри частицы $k_{\text{int}} = m \cdot k_{\text{ext}}$ тоже комплексно. Функция Грина $G_{k_{\text{int}}}$ экспоненциально затухает, что улучшает обусловленность внутренних операторов.

Закон сохранения энергии:

$$Q_{\text{abs}} = Q_{\text{ext}} - Q_{\text{sca}} \geq 0 \tag{11}$$

3 Дискретизация: RWG базисные функции

3.1 Определение

Базисные функции Рао–Уилтона–Глиссона [1] определены на парах треугольников T_n^+, T_n^-, разделяющих ребро длины l_n:

$$\mathbf{f}_n(\mathbf{r}) = \begin{cases} +\dfrac{l_n}{2A_n^+}(\mathbf{r} - \mathbf{r}_n^+), & \mathbf{r} \in T_n^+ \\ -\dfrac{l_n}{2A_n^-}(\mathbf{r} - \mathbf{r}_n^-), & \mathbf{r} \in T_n^- \end{cases} \tag{12}$$

где $\mathbf{r}_n^{\pm}$ — «свободные» вершины (не принадлежащие общему ребру), $A_n^{\pm}$ — площади треугольников.

Каждая внутренняя грань (общее ребро двух треугольников) порождает одну RWG-функцию. Число RWG-функций N для замкнутой поверхности из T треугольников и V вершин: $N = 3T/2 - V + 2 \approx 3T/2$.

3.2 Поверхностная дивергенция

$$\nabla_s \cdot \mathbf{f}_n = \begin{cases} +\dfrac{l_n}{A_n^+}, & \mathbf{r} \in T_n^+ \\ -\dfrac{l_n}{A_n^-}, & \mathbf{r} \in T_n^- \end{cases} \tag{13}$$

Важно: множитель $l_n/A_n^{\pm}$, а НЕ $l_n/(2A_n^{\pm})$. Фактор 2 в знаменателе присутствует только в самих базисных функциях $\mathbf{f}_n$, но не в их дивергенции. Это следует из того, что $\nabla_s \cdot (\mathbf{r} - \mathbf{r}_0) = 2$ на плоском треугольнике.

3.3 Масштаб задачи: icosphere

Функция `icosphere(radius, refinements)` генерирует сферическую сетку рекурсивным разбиением икосаэдра:

Refinements	Треугольники	Вершины	RWG	Рекомендация
0	20	12	30	отладка
1	80	42	120	$ka < 0.5$
2	320	162	480	$ka \approx 1$, ∼5% ошибка
3	1280	642	1920	$ka \approx 1$–3, ∼1% ошибка
4	5120	2562	7680	$ka \approx 3$–6, <1% ошибка
5	20480	10242	30720	$ka \approx 6$–12

3.4 Выбор разрешения сетки

Правило: необходимо ∼10 элементов на длину волны $\lambda = 2\pi/k$. Для сферы с размерным параметром ka:

$$N_{\text{triangles}} \approx 30 \cdot (ka)^2, \qquad \texttt{refinements} \approx \lceil \log_4(30 \cdot ka^2) \rceil \tag{14}$$

Краткая рекомендация:

- ref=2 для $ka \leq 1$
- ref=3 для $ka \leq 3$
- ref=4 для $ka \leq 6$
- ref=5 для $ka \leq 12$

Для несферических частиц используйте `adaptive_refine()` (раздел 5) — она автоматически подберёт разрешение.

4 Загрузка произвольных сеток

Функция `load_mesh(filename)` загружает сетку из файла. Поддерживаемые форматы:

Формат	Расширение	Особенности
STL	`.stl`	Binary и ASCII. Дубликаты вершин сливаются автоматически
OBJ	`.obj`	Поддержка `v/vt/vn`. Квады разбиваются на 2 треугольника
Gmsh	`.msh`	Версии 2 и 4. Извлекаются элементы типа 2 (треугольники)

Требование: сетка должна быть **замкнутой** (нет граничных рёбер). Генерация: Gmsh, MeshLab, любой CAD с экспортом в STL/OBJ.

```
from bem_core import load_mesh, build_rwg
verts, tris = load_mesh("particle.stl")
rwg = build_rwg(verts, tris)
print(f"{len(tris)} triangles, {rwg['N']} RWG")
```

5 Адаптивное уточнение сетки

5.1 Red-refinement с green closure

Функция `refine_mesh(verts, tris, mask)` реализует конформное адаптивное уточнение:

- **Red refinement**: отмеченные треугольники ($\texttt{mask}[i] = \texttt{True}$) разбиваются на 4 подтреугольника вставкой средних точек на каждом ребре.
- **Green closure**: соседние неотмеченные треугольники, разделяющие разбитое ребро, делятся на 2 или 3 части для сохранения конформальности (отсутствия висячих узлов).
- При $\texttt{mask} = \texttt{None}$ — равномерное уточнение.
- $\texttt{project_to_sphere} = \texttt{True}$ — проецирует новые вершины на сферу.

5.2 Автоматическое уточнение по длине волны

Функция `adaptive_refine(verts, tris, k, max_edge_per_wavelength=0.2)` итеративно уточняет сетку:

$$h_{\max} \leq \texttt{max_edge_per_wavelength} \times \lambda, \qquad \lambda = \frac{2\pi}{|k|} \tag{15}$$

По умолчанию `max_edge_per_wavelength = 0.2` ($\lambda/5$) — стандартный критерий BEM.

```python
from bem_core import icosphere, adaptive_refine, build_rwg

#
verts, tris = icosphere(1.0, refinements=1)

#                                                    lambda/5
verts, tris = adaptive_refine(verts, tris, k=6.0,
    max_edge_per_wavelength=0.2,
    project_to_sphere=True, sphere_radius=1.0)
rwg = build_rwg(verts, tris)
```

6 Сборка матрицы: операторы L и K

6.1 Дискретный оператор L

Элементы матрицы L методом Галёркина с RWG тестовыми и базисными функциями:

$$\boxed{L_{mn} = ik \int_S \int_S \mathbf{f}_m \cdot \mathbf{f}_n \, G_k \, dS \, dS' \; - \; \frac{i}{k} \int_S \int_S (\nabla \cdot \mathbf{f}_m)(\nabla' \cdot \mathbf{f}_n) \, G_k \, dS \, dS'} \tag{16}$$

Критический знак: перед div-div членом стоит **минус**. Это следует из представления рассеянного поля:

$$\mathbf{E}^s = -i\omega \mathbf{A} - \nabla \Phi \tag{17}$$

Минус перед $\nabla\Phi$ даёт минус перед div-div. Плюс — распространённая ошибка в литературе.

Каждый элемент L_{mn} включает 4 двойных интеграла по парам полу-треугольников: (T_m^+, T_n^+), (T_m^+, T_n^-), (T_m^-, T_n^+), (T_m^-, T_n^-).

6.2 Дискретный оператор K

$$K_{mn} = \int_S \int_S \mathbf{f}_m \cdot \left[\nabla G_k \times \mathbf{f}_n \right] dS \, dS' \tag{18}$$

Градиент функции Грина:

$$\nabla G_k(R) = \frac{e^{ikR}}{4\pi R} \left(ik - \frac{1}{R} \right) \frac{\mathbf{R}}{R} \tag{19}$$

Тройное произведение $\mathbf{f}_m \cdot (\nabla G \times \mathbf{f}_n)$ вычисляется через BLAS DGEMM для ускорения.

6.3 Эксплуатация симметрии (3 прохода вместо 4)

Матрицы L и K симметричны: $L_{mn} = L_{nm}$, $K_{mn} = K_{nm}$. Используется для ускорения:

$$L = L_{\text{pp}} + L_{\text{mm}} + L_{\text{pm}} + L_{\text{pm}}^T \tag{20}$$

Три прохода (pp, mm, pm) вместо четырёх (pp, mm, pm, mp). Ускорение ~25%.

6.4 Батчевая сборка

Для экономии памяти тестовые функции обрабатываются батчами по B штук. Размер батча автоматически подбирается для ограничения потребления памяти ~2 ГБ. Каждый батч использует BLAS DGEMM для матричных произведений.

7 Обработка сингулярностей

7.1 Извлечение сингулярности (Graglia 1993) [2,14]

При совпадении треугольников ($T_m = T_n$) функция Грина имеет интегрируемую особенность $1/R$. Разбиваем:

$$G_k(R) = \underbrace{\frac{1}{4\pi R}}_{G_0 \text{ (сингулярная)}} + \underbrace{\frac{e^{ikR} - 1}{4\pi R}}_{G_{\text{smooth}} \text{ (гладкая)}} \tag{21}$$

$G_{\text{smooth}}(0) = ik/(4\pi)$ — конечна, интегрируется стандартной квадратурой. G_0 — интегрируется аналитически [14,15].

7.2 Аналитический интеграл потенциала

Для G_0 вычисляется аналитически (Graglia 1993):

$$P(\mathbf{r}) = \int_T \frac{dS'}{|\mathbf{r} - \mathbf{r}'|} \tag{22}$$

через сумму по рёбрам треугольника с использованием логарифмов и арктангенсов.

Также вычисляется векторный потенциал:

$$\mathbf{V}(\mathbf{r}) = \int_T \frac{\mathbf{r}'}{|\mathbf{r} - \mathbf{r}'|}\, dS' = \mathbf{r} \cdot P(\mathbf{r}) - \nabla_r W(\mathbf{r}) \tag{23}$$

где $W = \int_T |\mathbf{r} - \mathbf{r}'|\, dS'$ вычисляется квадратурой, $\nabla_r W$ — конечными разностями.

7.3 Оператор K: PV-интеграл

Для копланарных треугольников ($T_m = T_n$) главное значение интеграла $\mathcal{K}$ равно нулю по симметрии. Сингулярные пары пропускаются в цикле сборки.

7.4 Квадратуры Dunavant

Функция `tri_quadrature(order)` возвращает точки и веса квадратуры Dunavant [5] на единичном треугольнике:

`order`	Точек	Алг. степень	Когда использовать
4	4	3	Быстрые оценки
7	7	5	**По умолчанию.** Достаточно для $ka \leq 3$
13	13	7	$ka = 3$–6, или повышенная точность
25	25	10	$ka > 6$, или точность $< 0.1\%$

Рекомендация: для большинства задач `quad_order=7` достаточно. Ошибка BEM определяется разрешением сетки, а не квадратурой. Увеличивать порядок квадратуры имеет смысл только если сетка уже достаточно мелкая.

Веса нормированы так, что $\sum w_i = 1$ (полный единичный треугольник). В коде площадь треугольника A входит как множитель в квадратурные веса: $\mathrm{jw} = A \cdot w_q$.

8 Сборка PMCHWT системы

Функция `assemble_pmchwt(rwg, verts, tris, k_ext, k_int, eta_ext, eta_int)` собирает блочную систему размера $2N \times 2N$:

$$Z = \begin{pmatrix} \eta_1 L_1 + \eta_2 L_2 & -(K_1 + K_2) \\ K_1 + K_2 & L_1/\eta_1 + L_2/\eta_2 \end{pmatrix} \tag{24}$$

8.1 Параллельная сборка

Параметр `parallel=True` (по умолчанию) запускает сборку внешних (L_1, K_1) и внутренних (L_2, K_2) операторов в параллельных потоках. BLAS отпускает GIL, поэтому два потока Python эффективно параллелизуются. Ускорение $\sim$1.5$\times$.

8.2 Правая часть

Функция `compute_rhs_planewave(rwg, verts, tris, k_ext, eta_ext, E0, k_hat)`:

$$b_E[n] = \int_S \mathbf{f}_n \cdot \mathbf{E}^{\mathrm{inc}}\, dS \tag{25}$$

$$b_H[n] = \int_S \mathbf{f}_n \cdot \mathbf{H}^{\mathrm{inc}}\, dS \tag{26}$$

Вектор $b = [b_E; b_H]$ имеет размерность $2N$.

8.3 Пример: полный цикл решения

```python
import numpy as np
from bem_core import (icosphere, build_rwg, assemble_pmchwt,
                      compute_rhs_planewave, compute_cross_sections
                         )

#
radius = 1.0; m_rel = 1.5; ka = 2.0
k_ext = ka / radius
k_int = k_ext * m_rel
eta_ext = 1.0; eta_int = eta_ext / m_rel

#
verts, tris = icosphere(radius, refinements=3)
rwg = build_rwg(verts, tris)
N = rwg['N']

#
```

```
Z, L_ext, K_ext = assemble_pmchwt(
    rwg, verts, tris, k_ext, k_int, eta_ext, eta_int)
b = compute_rhs_planewave(rwg, verts, tris, k_ext, eta_ext,
    E0=np.array([1,0,0.]), k_hat=np.array([0,0,1.]))
coeffs = np.linalg.solve(Z, b)
J, M = coeffs[:N], coeffs[N:]

#
Q_ext, Q_sca = compute_cross_sections(
    rwg, verts, tris, J, M, k_ext, eta_ext, radius, sM=-1)
Q_abs = Q_ext - Q_sca
print(f"Q_ext={Q_ext:.4f}, Q_sca={Q_sca:.4f}, Q_abs={Q_abs:.4f}")
```

9 Multi-body BEM (кластеры частиц)

Поддержка рассеяния на кластерах из нескольких частиц, каждая со своим показателем преломления и геометрией.

9.1 Принцип

Для кластера из P частиц строится единая PMCHWT-система:

$$Z = \underbrace{Z_{\text{ext}}}_{\text{полная } N \times N \text{ связь}} + \underbrace{Z_{\text{int}}}_{\text{блочно-диагональная}} \tag{27}$$

- **Внешние операторы** $(L_{\text{ext}}, K_{\text{ext}})$: собираются на объединённой сетке всех частиц с единым k_{ext}. Обеспечивают электромагнитную связь между телами.
- **Внутренние операторы** $(L_{\text{int}}^{(i)}, K_{\text{int}}^{(i)})$: блочно-диагональные, каждое тело собирается отдельно со своими $k_{\text{int}}^{(i)}, \eta_{\text{int}}^{(i)}$.

9.2 API

```
from bem_core import icosphere, assemble_multibody_pmchwt
from bem_core import compute_rhs_planewave

#                                        m
v1, t1 = icosphere(0.5, refinements=2)
v2, t2 = icosphere(0.5, refinements=2)
v2[:, 0] += 3.0  #                        x

bodies = [
    {'verts': v1, 'tris': t1,
     'k_int': k_ext * 1.5, 'eta_int': eta_ext / 1.5},
    {'verts': v2, 'tris': t2,
     'k_int': k_ext * 1.3, 'eta_int': eta_ext / 1.3},
]

Z, rwg, verts, tris, body_rwg_ranges = \
```

```
    assemble_multibody_pmchwt(bodies, k_ext, eta_ext)
b = compute_rhs_planewave(rwg, verts, tris, k_ext, eta_ext,
    E0=np.array([1,0,0.]), k_hat=np.array([0,0,1.]))
coeffs = np.linalg.solve(Z, b)
```

10 Решение системы

10.1 Прямой метод: LU-разложение

`numpy.linalg.solve` или `scipy.linalg.lu_factor/lu_solve` [13]. Сложность $O(N^3)$.

Преимущество: при усреднении по ориентациям LU факторизуется один раз ($O(N^3)$), каждая новая правая часть решается за $O(N^2)$. Для 1000 ориентаций это в 1000 раз дешевле, чем 1000 полных решений.

10.2 GMRES (итерационный)

Функция `solve_gmres(Z, b, tol, maxiter, precond)` — GMRES с блочно-диагональным предобусловливателем:

$$M^{-1} = \begin{pmatrix} Z_{JJ}^{-1} & 0 \\ 0 & Z_{MM}^{-1} \end{pmatrix} \tag{28}$$

Каждый блок $(N \times N)$ факторизуется LU один раз. Для PMCHWT GMRES сходится за 10–20 итераций. Matvec $O(N^2)$, полная стоимость $O(N^2 \cdot n_{\text{iter}})$ — выигрыш при $n_{\text{iter}} \ll N$.

10.3 H-матричный GMRES

Класс `HMatrix` сжимает матрицу Z с помощью ACA [22,23] (Adaptive Cross Approximation):

- Иерархическая бисекция центроидов RWG-функций
- Дальнеполевые блоки: низкоранговое UV^H через ACA с partial pivoting
- Ближнеполевые блоки: плотные
- Matvec: $O(N \log N)$ вместо $O(N^2)$

```
from bem_core import HMatrix, solve_hmatrix_gmres

Z_hmat = HMatrix(Z, rwg, verts, tris,
                 eta=3.0, aca_tol=1e-4, max_rank=50)
print(f"Memory: {Z_hmat.memory_bytes()/1e6:.1f} MB")
x = solve_hmatrix_gmres(Z_hmat, b, tol=1e-6,
    Z_diag_blocks=[Z[:N,:N], Z[N:,N:]])
```

Параметры H-матрицы:

Параметр	Значение	Описание
`eta`	3.0	Допустимость: блок дальнеполевой если $d > \eta \cdot \max(\mathrm{diam}_1, \mathrm{diam}_2)$
`aca_tol`	10^{-4}	Точность ACA-аппроксимации
`max_rank`	50	Максимальный ранг низкоранговых блоков

Результаты ($N = 1920$): 15 итераций GMRES, ошибка 6×10^{-7}, сжатие 93.7%.

10.4 FMM-ускоренный GMRES

Класс `FMMOperator` [7,19,20,30] — октодеревная версия:

1. Построение октодерева по центроидам RWG-функций
2. Разбиение пар листьев на ближне- и дальнеполевые
3. Дальнеполевые блоки: SVD-сжатие с обрезкой
4. Matvec через сумму плотных и низкоранговых блоков

```
from bem_core import solve_fmm_gmres

x = solve_fmm_gmres(rwg, verts, tris, Z, b,
    k_ext, eta_ext, tol=1e-6, maxiter=200)
```

Результаты ($N = 1920$): 8 итераций, ошибка 3.6×10^{-5}.

11 Дальнее поле и сечения

11.1 Амплитуда рассеяния

$$\mathbf{F}(\hat{\mathbf{r}}) = -\frac{ik}{4\pi}\left[\eta\,\tilde{\mathbf{J}}_\perp + s_M\,\hat{\mathbf{r}} \times \tilde{\mathbf{M}}\right] \tag{29}$$

где $\tilde{\mathbf{J}} = \int_S \mathbf{J}\, e^{-ik\hat{\mathbf{r}}\cdot\mathbf{r}'} dS'$ — Фурье-образ тока, $\tilde{\mathbf{J}}_\perp = \tilde{\mathbf{J}} - \hat{\mathbf{r}}(\hat{\mathbf{r}} \cdot \tilde{\mathbf{J}})$ — его проекция на плоскость, перпендикулярную направлению наблюдения.

Знак $s_M = -1$ для PMCHWT-конвенции. Для PEC $\mathbf{M} = 0$, знак не важен.

11.2 Сечения рассеяния и экстинкции

$$C_{\mathrm{sca}} = \int_{4\pi} \frac{|\mathbf{F}|^2}{|E_0|^2}\, d\Omega \qquad \Rightarrow \qquad Q_{\mathrm{sca}} = \frac{C_{\mathrm{sca}}}{\pi a^2} \tag{30}$$

$$Q_{\mathrm{ext}} = \frac{4\pi}{k\pi a^2}\,\mathrm{Im}\left[\mathbf{F}(\hat{\mathbf{k}}) \cdot \hat{\mathbf{e}}_{\mathrm{pol}}\right] \quad \text{(оптическая теорема)} \tag{31}$$

Интегрирование C_{sca} по θ, ϕ с учётом ϕ-зависимости:

$$C_{\mathrm{sca}} = \pi \int_0^\pi \left[|F_\theta(\theta, 0)|^2 + |F_\phi(\theta, \pi/2)|^2\right] \sin\theta\, d\theta \tag{32}$$

11.3 Поглощение

Для поглощающих частиц ($\mathrm{Im}(m) > 0$):

$$Q_{\mathrm{abs}} = Q_{\mathrm{ext}} - Q_{\mathrm{sca}} \geq 0 \tag{33}$$

12 Матрица амплитудного рассеяния и матрица Мюллера

12.1 Амплитудная матрица

Связь компонент рассеянного поля с падающим (конвенция Борена–Хаффмана [4, глава 3]):

$$\begin{pmatrix} E_{\parallel}^{s} \\ E_{\perp}^{s} \end{pmatrix} = \frac{e^{ikr}}{-ikr} \begin{pmatrix} S_2 & S_3 \\ S_4 & S_1 \end{pmatrix} \begin{pmatrix} E_{\parallel}^{i} \\ E_{\perp}^{i} \end{pmatrix} \tag{34}$$

Связь с дальним полем: $S_j = -ik \cdot F_j$. Для осесимметричных тел с падением вдоль оси симметрии: $S_3 = S_4 = 0$.

Функция `compute_amplitude_matrix` решает систему для двух поляризаций (par и perp) и возвращает $S_1(\theta), S_2(\theta), S_3(\theta), S_4(\theta)$.

12.2 Матрица Мюллера

4×4 матрица, связывающая параметры Стокса [4, eq. 3.16; 25, глава 2]:

$$M_{11} = \tfrac{1}{2}(|S_1|^2 + |S_2|^2 + |S_3|^2 + |S_4|^2) \tag{35}$$

$$M_{12} = \tfrac{1}{2}(|S_2|^2 - |S_1|^2 + |S_4|^2 - |S_3|^2) \tag{36}$$

$$M_{33} = \mathrm{Re}(S_1 S_2^* + S_3 S_4^*) \tag{37}$$

$$M_{34} = \mathrm{Im}(S_2 S_1^* + S_4 S_3^*) \tag{38}$$

Нормализация: M_{ij}/k^2, так что $\int M_{11}\, d\Omega = C_{\mathrm{sca}}$.

Конвенция ADDA [6]: ADDA выводит S_{ij} без деления на k^2. Для сравнения с BEM/Mie нужно умножить на $1/k_{\mathrm{ext}}^2 = 1/(ka)^2$.

12.3 Пример: вычисление матрицы Мюллера

```
from scipy.linalg import lu_factor
from bem_core import (assemble_pmchwt, compute_mueller_matrix)

Z, _, _ = assemble_pmchwt(rwg, verts, tris,
    k_ext, k_int, eta_ext, eta_int)
Z_lu = lu_factor(Z)

theta = np.linspace(0.01, np.pi - 0.01, 181)
M = compute_mueller_matrix(rwg, verts, tris, k_ext, eta_ext,
    theta, Z_lu=Z_lu, sM=-1)
# M[i,j,:] --                    (i,j)
# M[0,0,:] --
```

13 Усреднение по ориентациям

13.1 Теория

Для случайно ориентированных частиц средняя матрица Мюллера [25, глава 5]:

$$\langle M_{ij}(\theta)\rangle = \frac{1}{8\pi^2}\int_0^{2\pi} d\alpha \int_0^{\pi} \sin\beta\, d\beta \int_0^{2\pi} d\gamma\; M_{ij}(\theta;\alpha,\beta,\gamma) \tag{39}$$

Углы Эйлера (α, β, γ) в конвенции ZYZ. Квадратура:

- α: равномерная на $[0, 2\pi)$, n_α точек
- β: Гаусса–Лежандра на $[0, \pi]$, n_β точек (с весом $\sin\beta$)
- γ: равномерная на $[0, 2\pi)$, n_γ точек

Общее число ориентаций: $n_\alpha \times n_\beta \times n_\gamma$. Для каждой — 2 решения (2 поляризации).

13.2 Почему матрицу Z не нужно пересобирать

Ключевое наблюдение: матрица Z зависит только от **геометрии частицы** (форма, m, k), но **не от направления падения** $\hat{\mathbf{k}}$ и поляризации $\mathbf{E}_0$. Направление и поляризация входят только в правую часть $\mathbf{b}$.

Поэтому при усреднении по N_{or} ориентациям:

- **Без оптимизации**: N_{or} раз решить $Z\mathbf{x} = \mathbf{b}_i$ (каждый раз $O(N^3)$ LU) $\Rightarrow$ общая стоимость $O(N_{\text{or}} \cdot N^3)$.
- **С LU-факторизацией**: факторизовать $Z = PLU$ **один раз** ($O(N^3)$), затем для каждой ориентации — только обратная подстановка $O(N^2)$. Общая стоимость: $O(N^3 + N_{\text{or}} \cdot N^2)$.

При $N_{\text{or}} = 1000$ и $N = 2000$ это ускорение в $\sim$1000 раз на этапе решения.

13.3 Наивная vs батчевая реализация

Наивная версия (`orientation_average_mueller`):

1. Цикл по N_{or} ориентациям в Python
2. Для каждой: собрать $\mathbf{b}_i$ $\rightarrow$ `lu_solve(Z_lu, b_i)` $\rightarrow$ посчитать дальнее поле $\rightarrow$ матрицу Мюллера
3. Каждый вызов `lu_solve` решает систему $2N \times 2N$ для одного вектора

Батчевая версия (`orientation_average_mueller_batched`):

1. Сначала собрать **все** правые части в матрицу B размера $2N \times 2N_{\text{or}}$:

$$B = [\mathbf{b}_1^{\parallel}, \mathbf{b}_1^{\perp}, \mathbf{b}_2^{\parallel}, \mathbf{b}_2^{\perp}, \ldots, \mathbf{b}_{N_{\text{or}}}^{\parallel}, \mathbf{b}_{N_{\text{or}}}^{\perp}] \tag{40}$$

(по 2 столбца на ориентацию — две поляризации)

2. **Один вызов** `lu_solve(Z_lu, B)` — LAPACK решает $2N_{\text{or}}$ систем за раз

3. Затем цикл по ориентациям только для дальнего поля (лёгкие операции)

Почему батч быстрее? LAPACK-функция `dtrsv` (один вектор) работает с памятью побайтово — ограничена пропускной способностью памяти (memory-bound). Функция `dtrsm` (матрица правых частей) обрабатывает блоки столбцов, используя кеш-линии и BLAS Level-3 (матрица×матрица вместо матрица×вектор). Соотношение вычислений к обращениям к памяти растёт с $O(1)$ до $O(B)$, где B — число столбцов в блоке.

Результат: шаг решения ускоряется в $\sim$9.4× при 1024 ориентациях ($N = 1920$). Общее ускорение меньше, т.к. сборка правых частей и дальнее поле остаются последовательными.

```
from bem_core import orientation_average_mueller_batched

M_avg = orientation_average_mueller_batched(
    rwg, verts, tris, k_ext, eta_ext, theta,
    Z_lu=Z_lu, sM=-1,
    n_alpha=16, n_beta=8, n_gamma=8)
# 16 * 8 * 8 = 1024
```

Для сферы: $\langle M_{ij} \rangle = M_{ij}$ (тест: отклонение $< 10^{-4}$).

14 OpenCL-ускорение

Модуль `bem_opencl.py` содержит float32 ядра для GPU/CPU OpenCL [29].

14.1 Зачем float32?

Потребительские GPU (AMD RDNA 4, NVIDIA GeForce) имеют fp64 производительность 1/16–1/32 от fp32. Для BEM с точностью 1–3% потеря 7 значащих цифр не критична.

14.2 Ядро сборки `assemble_LK_block`

Один work-item вычисляет один элемент (b, n) матриц L и K:

- Внутренний цикл по $N_q \times N_q$ парам квадратурных точек
- Функция Грина, скалярное произведение, тройное произведение — всё в регистрах
- Поддержка всех `quad_order`: 4, 7, 13, 25 (N_q передаётся как аргумент)
- Сингулярные пары обрабатываются отдельно: G_{smooth} вместо G

Результат float32 конвертируется в float64 на хосте. Относительная разница с CPU float64: $\sim 10^{-5}$–10^{-7}.

14.3 Ядро matvec `complex_matvec`

$y = Z \cdot x$ на GPU для GMRES. Матрица Z хранится в float32 (re/im отдельно) — 2× меньше памяти.

14.4 Сингулярные коррекции на CPU

Аналитические интегралы (Graglia [2]) вычисляются на CPU в float64. При наличии Numba [30] — JIT-компилированная версия с `prange`: ∼2.5 с → ∼0 с.

14.5 Производительность (ref=3, 1920 RWG)

Этап	NumPy (CPU)	POCL (CPU OpenCL)	GPU (оценка)
Сборка (основной цикл)	68 с	3 с	∼0.1 с
Сингулярные коррекции	2.5 с (Numba: 0 с)	2.5 с	2.5 с
GMRES solve	7 с	2.5 с	∼0.3 с
Итого	**87 с**	**12.6 с**	**∼3 с**

14.6 Установка OpenCL

```
pip install pyopencl

# CPU OpenCL (        GPU):
sudo apt-get install pocl-opencl-icd

# AMD GPU (ROCm):
sudo apt-get install rocm-opencl-runtime

# NVIDIA GPU:
sudo apt-get install nvidia-opencl-icd

#                  :
python -c "import pyopencl; print(pyopencl.get_platforms())"
```

14.7 Использование

```
from bem_opencl import assemble_pmchwt_ocl, solve_gmres_ocl
from bem_core import compute_rhs_planewave

Z = assemble_pmchwt_ocl(rwg, verts, tris,
    k_ext, k_int, eta_ext, eta_int, quad_order=7)
b = compute_rhs_planewave(rwg, verts, tris, k_ext, eta_ext)
coeffs = solve_gmres_ocl(Z, b, tol=1e-6, maxiter=200)
```

15 Валидация

15.1 Диэлектрическая сфера vs Mie ($m = 1.5$, реальный)

ka	ref	N_{RWG}	Ошибка Q_{ext}	Время (CPU)
1.0	2	480	5.9%	10 с
1.0	3	1920	1.4%	87 с
2.0	3	1920	0.66%	87 с
3.0	3	1920	1.7%	87 с

15.2 PEC сфера vs Mie

ka	ref	N_{RWG}	Ошибка Q_{sca}
1.0	1	120	15.7%
1.0	2	480	4.3%
1.0	3	1920	1.2%

15.3 Поглощающие частицы ($m \in \mathbb{C}$)

Частица	m	ka	Ошибка Q_{ext}	Ошибка Q_{sca}
Слабое поглощение	$1.5 + 0.01i$	2.0	0.7%	0.7%
Умеренное	$1.5 + 0.1i$	2.0	0.7%	0.7%
Сильное	$1.5 + 0.5i$	2.0	0.7%	0.8%
Металлоподобное	$1.33 + 1.0i$	1.0	0.8%	1.2%
Капля воды	$1.33 + 10^{-6}i$	2.0	0.6%	0.6%

Все ошибки $< 1.2\%$ при ref=3. $Q_{\text{abs}} = Q_{\text{ext}} - Q_{\text{sca}} \geq 0$ — физически корректно.

15.4 Оптическая теорема

$Q_{\text{ext}} = Q_{\text{sca}}$ для PEC и реального m (нет поглощения) — выполняется с точностью $< 0.1\%$.

15.5 Матрица Мюллера

Для сферы при $ka = 1$, ref=3:

Элемент	RMS ошибка vs Mie
M_{11}/M_{11}	1.43%
M_{12}/M_{11}	0.79%
M_{33}/M_{11}	1.21%
M_{34}/M_{11}	0.00% (нуль для сферы)

Проверки симметрии для сферы:

- $S_3 = S_4 \approx 10^{-14}$ (ноль)
- $M_{22} = M_{33}$, $M_{23} = -M_{32}$ — точно
- Блочно-диагональная структура (внедиагональные $\sim 10^{-14}$)

15.6 Несферические частицы

Форма	Q_{ext}	Q_{sca}	Q_{abs}	Статус
Пролатный сфероид (2:1)	0.518	0.518	0.0002	PASS
Облатный сфероид (1:2)	0.072	0.072	$\sim$0	PASS
Куб	0.036	0.036	$\sim$0	PASS

Для сфероида с падением вдоль оси симметрии: $S_3, S_4 \sim 10^{-13}$ (машинный ноль), матрица Мюллера блочно-диагональная.

16 Масштабирование BEM vs DDA

	BEM (PMCHWT + LU)	DDA (ADDA + FFT)
Неизвестные	$N \sim (ka)^2$ (поверхность)	$N \sim (ka)^3$ (объём)
Сборка	$O(N^2)$	$O(N)$
Решение (LU / FFT)	$O(N^3)$	$O(N \log N)$
Память	$O(N^2)$	$O(N)$

BEM выгоднее когда:

- **Много правых частей** (усреднение по ориентациям): LU факторизуется один раз, каждое решение — $O(N^2)$. При 1000 ориентациях BEM быстрее DDA.
- **Тонкие/плоские частицы**: поверхностных элементов меньше объёмных ($N_{\text{surf}} \sim L^2$ vs $N_{\text{vol}} \sim L^2 \cdot t$).
- **С FMM/H-матрицами**: $O(N \log N)$ сборка и matvec, сравнимо с DDA.
- **Малые** ka: $N \sim (ka)^2 \ll (ka)^3$, BEM экономичнее.

DDA выгоднее когда [27,28]:

- Большие $ka > 10$: N^2 память BEM становится неподъёмной
- Неоднородные частицы (градиент m внутри)
- Одна ориентация, одна поляризация

17 Исправленные ошибки

В процессе разработки были найдены и исправлены 4 критические ошибки. Каждая давала 10–50% погрешности; совместно — $\sim$50% ошибку.

1. **Знак div-div члена в** L: было $+i/k \cdot D$, исправлено на $-i/k \cdot D$. Следует из $E^s = -i\omega A - \nabla\Phi$.
2. **Множитель дивергенции**: было $l/(2A)$, исправлено на l/A. Фактор 2 присутствует в $\mathbf{f}_n$, но не в $\nabla \cdot \mathbf{f}_n$.

3. **Знаки K-блоков в PMCHWT**: было $[+K; -K]$, исправлено на $[-K; +K]$ с $+\mathbf{b}$.

4. **Знак s_M в дальнем поле**: было $s_M = +1$, исправлено на $s_M = -1$ для PMCHWT.

После исправлений: 0.66% при $ka = 2$, ref=3.

18 Ограничения и перспективы

18.1 Текущие ограничения

1. **Только замкнутые поверхности**: меш должен быть замкнутым (нет граничных рёбер).

2. **Плотная матрица по умолчанию**: $O(N^2)$ память, $O(N^3)$ LU. Для $N > 5000$ используйте FMM или H-матрицы.

3. **Нет адаптивной квадратуры**: для близких (но не совпадающих) треугольников используется та же квадратура, что и для дальних.

4. **Float32 OpenCL**: 7 значащих цифр; достаточно для BEM (1–3%), но не для прецизионных вычислений.

5. **GPU в WSL2**: AMD GPU passthrough пока не поддерживается (ограничение драйверов).

18.2 Что реализовано (бывшие ограничения)

✓ Множественные тела — `assemble_multibody_pmchwt`

✓ Комплексный m — валидация, ошибки $< 1.2\%$

✓ Адаптивная сетка — `adaptive_refine`

✓ Высокие квадратуры — Dunavant 13, 25 точек

✓ Ускоренные солверы — FMM, H-матрицы, GMRES

19 Литература

19.1 Основные источники

1. S.M. Rao, D.R. Wilton, A.W. Glisson, “Electromagnetic scattering by surfaces of arbitrary shape”, *IEEE Trans. Antennas Propag.*, vol. 30, no. 3, pp. 409–418, 1982. *Оригинальная статья RWG базисных функций. Определение базиса, EFIE для PEC, численные результаты.*

2. R.D. Graglia, “On the numerical integration of the linear shape functions times the 3-D Green’s function or its gradient on a plane triangle”, *IEEE Trans. Antennas Propag.*, vol. 41, no. 10, pp. 1448–1455, 1993.
Аналитическое вычисление сингулярных интегралов $\int 1/R\, dS'$ на треугольнике. Используется в нашей реализации для извлечения сингулярности.

3. A.J. Poggio, E.K. Miller, “Integral equation solutions of three-dimensional scattering problems”, in *Computer Techniques for Electromagnetics*, R. Mittra, Ed., Pergamon, 1973.
Формулировка PMCHWT для диэлектрических тел. Условие непрерывности тангенциальных компонент E и H.

4. C.F. Bohren, D.R. Huffman, *Absorption and Scattering of Light by Small Particles*, Wiley, 1983.
Теория Ми, матрица Мюллера, параметры Стокса. Главы 3–4 — основа нашей конвенции для амплитудной и мюллеровской матриц.

5. D.A. Dunavant, “High degree efficient symmetrical Gaussian quadrature rules for the triangle”, *Int. J. Numer. Methods Eng.*, vol. 21, pp. 1129–1148, 1985.
Квадратурные формулы на треугольнике степеней 1–20. Мы используем правила 4, 7, 13, 25 точек.

6. M.A. Yurkin, A.G. Hoekstra, “The discrete-dipole-approximation code ADDA: capabilities and known limitations”, *J. Quant. Spectrosc. Radiat. Transfer*, vol. 112, pp. 2234–2247, 2011.
Описание кода ADDA для DDA. Сравнение нормализации матрицы Мюллера с BEM.

7. W.C. Chew, J.-M. Jin, E. Michielssen, J. Song, *Fast and Efficient Algorithms in Computational Electromagnetics*, Artech House, 2001.
FMM и быстрые алгоритмы для интегральных уравнений электромагнетизма. Главы 6–7 — мультипольные разложения.

8. S. Börm, L. Grasedyck, W. Hackbusch, “Hierarchical matrices”, *Lecture Notes*, Max-Planck-Institut für Mathematik, 2003.
H-матрицы и ACA (Adaptive Cross Approximation). Теория допустимости блоков и низкоранговой аппроксимации.

19.2 PMCHWT и интегральные уравнения

9. Y. Chang, R.F. Harrington, “A surface formulation for characteristic modes of material bodies”, *IEEE Trans. Antennas Propag.*, vol. 25, no. 6, pp. 789–795, 1977.
Развитие PMCHWT-формулировки. Определение характеристических мод для диэлектрических тел.

10. T.K. Wu, L.L. Tsai, “Scattering from arbitrarily-shaped lossy dielectric bodies of revolution”, *Radio Science*, vol. 12, no. 5, pp. 709–718, 1977.
PMCHWT для тел вращения с потерями. Комплексный m и поглощающие частицы.

11. P. Ylä-Oijala, M. Taskinen, “Well-conditioned Müller formulation for electromagnetic scattering by dielectric objects”, *IEEE Trans. Antennas Propag.*, vol. 53, no. 10, pp. 3316–3323, 2005.
Анализ обусловленности PMCHWT и альтернативная формулировка М¨ллера. Полезно для понимания знаков в матрице системы.

12. *P. Ylä-Oijala, M. Taskinen, “Calculation of CFIE impedance matrix elements with RWG and $\hat{n}\times$RWG functions”, IEEE Trans. Antennas Propag., vol. 51, no. 8, pp. 1837–1846, 2003.*
Детальное описание вычисления элементов матриц L и K с RWG-функциями [1]. Формулы для скалярных и векторных потенциалов.

13. *R.F. Harrington, Field Computation by Moment Methods, Macmillan, 1968 (переиздание IEEE Press, 1993).*
Классический учебник по методу моментов (MoM). Основы дискретизации интегральных уравнений методом Галёркина.

19.3 Сингулярные интегралы и квадратуры

9. D.R. Wilton, S.M. Rao, A.W. Glisson, D.H. Schaubert, O.M. Al-Bundak, C.M. Butler, “Potential integrals for uniform and linear source distributions on polygonal and polyhedral domains”, *IEEE Trans. Antennas Propag.*, vol. 32, no. 3, pp. 276–281, 1984.
Аналитические формулы потенциальных интегралов на треугольниках и многоугольниках. Дополнение к [2].

10. R.D. Graglia, G. Lombardi, “Machine precision evaluation of singular and nearly singular potential integrals by use of Gauss quadrature formulas for rational polynomial functions”, *IEEE Trans. Antennas Propag.*, vol. 56, no. 4, pp. 981–998, 2008.
Высокоточные квадратуры для почти сингулярных интегралов. Метод рациональных полиномов.

11. P. Arcioni, M. Bressan, L. Perregrini, “On the evaluation of the double surface integrals arising in the application of the boundary integral method to 3-D problems”, *IEEE Trans. Microw. Theory Tech.*, vol. 45, no. 3, pp. 436–439, 1997.
Методы вычисления двойных поверхностных интегралов BEM на близких треугольниках.

12. S. Sauter, C. Schwab, *Boundary Element Methods*, Springer, 2011.
Монография по BEM. Главы 4–5: квадратуры, сингулярные интегралы, оценки ошибок. Глава 7: H-матрицы.

19.4 Быстрые алгоритмы (FMM, H-матрицы)

13. V. Rokhlin, “Rapid solution of integral equations of scattering theory in two dimensions”, *J. Comput. Phys.*, vol. 86, pp. 414–439, 1990.
Оригинальная статья FMM. Алгоритм $O(N)$ для двумерных интегральных уравнений.

14. R. Coifman, V. Rokhlin, S. Wandzura, “The fast multipole method for the wave equation: a pedestrian prescription”, *IEEE Antennas Propag. Mag.*, vol. 35, no. 3, pp. 7–12, 1993.
Доступное введение в FMM для уравнения Гельмгольца. Мультипольные разложения и трансляционные операторы.

15. J. Song, C.C. Lu, W.C. Chew, “Multilevel fast multipole algorithm for electromagnetic scattering by large complex objects”, *IEEE Trans. Antennas Propag.*, vol. 45, no. 10, pp. 1488–1493, 1997.
Многоуровневый FMM (MLFMA) для задач рассеяния. Масштабирование $O(N \log N)$ для больших объектов.

16. M. Bebendorf, “Approximation of boundary element matrices”, *Numer. Math.*, vol. 86, pp. 565–589, 2000.
ACA (Adaptive Cross Approximation) — ядро нашей H-матричной реализации. Доказательство сходимости для ядер, удовлетворяющих условию допустимости.

17. M. Bebendorf, S. Rjasanow, “Adaptive low-rank approximation of collocation matrices”, *Computing*, vol. 70, pp. 1–24, 2003.
ACA с частичным пивотингом для матриц коллокации BEM. Практические аспекты реализации.

19.5 Рассеяние света, теория Ми, матрица Мюллера

18. G. Mie, “Beiträge zur Optik trüber Medien, speziell kolloidaler Metallösungen”, *Annalen der Physik*, vol. 330, no. 3, pp. 377–445, 1908.
Оригинальная работа Ми. Аналитическое решение для рассеяния на сфере.

19. H.C. van de Hulst, *Light Scattering by Small Particles*, Dover, 1981 (переиздание 1957).
Классический учебник по рассеянию света. Теория Ми, приближения Рэлея и геометрической оптики.

20. M.I. Mishchenko, L.D. Travis, A.A. Lacis, *Scattering, Absorption, and Emission of Light by Small Particles*, Cambridge University Press, 2002.
Современный учебник. Главы 2–5: T-матрица, матрица Мюллера, усреднение по ориентациям. Бесплатно на сайте NASA GISS.

21. M.I. Mishchenko, L.D. Travis, “T-matrix computations of light scattering by large spheroidal particles”, *Opt. Commun.*, vol. 109, pp. 16–21, 1994.
T-матричный метод для сфероидов. Валидационные данные для несферических частиц.

22. B.T. Draine, P.J. Flatau, “Discrete-dipole approximation for scattering calculations”, *J. Opt. Soc. Am. A*, vol. 11, no. 4, pp. 1491–1499, 1994.
Основы DDA. Сравнение с точными решениями. Критерий дипольного размера $|m|kd < 1$.

23. M.A. Yurkin, V.P. Maltsev, A.G. Hoekstra, “The discrete dipole approximation for simulation of light scattering by particles much larger than the wavelength”, *J.*

Quant. Spectrosc. Radiat. Transfer, vol. 106, pp. 546–557, 2007.
Масштабирование DDA для больших частиц. Сравнение с BEM по точности и времени.

19.6 Вычислительные аспекты и GPU

24. A. Klöckner, N. Pinto, Y. Lee, B. Catanzaro, P. Ivanov, A. Fasih, "PyCUDA and PyOpenCL: A scripting-based approach to GPU run-time code generation", *Parallel Computing*, vol. 38, no. 3, pp. 157–174, 2012.
Библиотека PyOpenCL, используемая в нашем модуле `bem_opencl.py`.

25. S.K. Lam, A. Pitrou, S. Seibert, "Numba: A LLVM-based Python JIT compiler", in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.
Numba — JIT-компилятор для Python. Используется для ускорения сингулярных коррекций.

26. T. Takahashi, T. Hirata, "GPU-accelerated boundary element method for Helmholtz' equation in three dimensions", *Int. J. Numer. Methods Eng.*, vol. 80, no. 10, pp. 1295–1321, 2009.
GPU-ускорение BEM для уравнения Гельмгольца. Стратегии параллелизации сборки матрицы.

27. L. Greengard, V. Rokhlin, "A fast algorithm for particle simulations", *J. Comput. Phys.*, vol. 73, no. 2, pp. 325–348, 1987.
Оригинальная статья быстрого мультипольного метода (FMM). Алгоритм $O(N)$ для N-body задач.